NORTH BENGAL INTERNATIONAL UNIVERSITY

Lab Report : 4

TOPIC : WRITE A PYTHON PROGRAM TO FIND THE ROOTS OF AN EQUATION OF BISECTION METHOD WHEN THE NUMBERS ARE INPUT BY THE USER

COURSE CODE : MATH 0541-2212

COURSE TITLE : NUMERICAL ANALYSIS SESSIONAL

Submitted By:

MD. TANVIR RAHMAN PRANTO

0712340012101005

CSE -26

Submitted To:

SAIFUR RAHMAN

LECTURER, CSE

Write a python program to find the roots of an equation of bisection method when the numbers are input by the user:

Theory:

A quadratic equation is a second-order polynomial equation in a single variable x with the general form ax^2 + bx + c = 0, where a, b, and c are constants, and a ≠ 0. The solutions to this equation, known as the roots, can be found using the quadratic formula:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The term inside the square root, b^2 - 4ac, is called the discriminant. It determines the nature of the roots:

- If the discriminant is positive (Δ > 0), the equation has two distinct real roots.
- If the discriminant is zero (Δ = 0), the equation has one real root (also called a repeated or double root).
- If the discriminant is negative (Δ < 0), the equation has no real roots but two complex conjugate roots.

The Python program provided uses user input for the coefficients a, b, and c to calculate the discriminant and find the real roots, if they exist. This approach demonstrates essential programming concepts such as input handling, conditional statements, and mathematical computations.

Algorithm:

A quadratic equation takes the form ax^2 + bx + c = 0, where a, b, and c are coefficients provided by the user. Solving such an equation involves finding the roots (values of x) where the equation equals zero. The program leverages Python's capabilities to handle these calculations and provide real solutions.

Steps:

1. Start.

2. Get the numbers:

- Ask the user to enter the values for a, b, and c .

3. Calculate a special number:

- Find the discriminant using the formula b^2 - 4ac. This special number helps us decide if there are real solutions.

4. Check the special number:

- If this number is not negative (0 or more):
  - Go to step 5.
- If this number is negative:
  - Print "The equation has no real solutions".
  - Go to step 8.

5. Find the solutions:

- Use two formulas to get the answers (roots):

- First solution: (-b ± √Δ) / 2a
- Second solution: (-b - √Δ) / 2a

6. Show the solutions:

- Print both solutions.

7. End.

Code:

```
# Function representing the equation
def f(x):
    return x**3 - x - 1  # Example equation

# Input values from the user
a = float(input("Enter the left endpoint a: "))
b = float(input("Enter the right endpoint b: "))
tol = 10**-4

# Bisection method function
def bisection(a, b, tol):
    if f(a) * f(b) >= 0:
        print("Bisection method fails.")
        return None
    steps = 0
    while (b - a) / 2.0 > tol:
        midpoint = (a + b) / 2.0
        print(f"Step {steps + 1}: midpoint = {midpoint}")
        if f(midpoint) == 0:
            return midpoint, steps
        elif f(a) * f(midpoint) < 0:
            b = midpoint
        else:
            a = midpoint
        steps += 1
    return (a + b) / 2.0, steps

# Finding the root
root, steps = bisection(a, b, tol)

if root is not None:
    print(f"The root is approximately: {root}")
    print(f"Total steps taken: {steps}")
else:
    print("Failed to find a root.")
```

Output:

```
OUTPUT    TERMINAL

PS C:\Users\Lenovo> & C:/Users/Lenovo/AppData/Local/Programs/Python/Python313/python.exe
Enter the left endpoint a: 1
Enter the right endpoint b: 2
Step 1: midpoint = 1.5
Step 2: midpoint = 1.25
Step 3: midpoint = 1.375
Step 4: midpoint = 1.3125
Step 5: midpoint = 1.34375
Step 6: midpoint = 1.328125
Step 7: midpoint = 1.3203125
Step 8: midpoint = 1.32421875
Step 9: midpoint = 1.326171875
Step 9: midpoint = 1.326171875
Step 9: midpoint = 1.326171875
Step 10: midpoint = 1.3251953125
Step 11: midpoint = 1.32470703125
Step 12: midpoint = 1.324951171875
Step 13: midpoint = 1.3248291015625
The root is approximately: 1.32476806640625
Total steps taken: 13
PS C:\Users\Lenovo>
```

Conclusion: This Python code demonstrates the bisection method to find the roots of an equation by iteratively narrowing the interval and printing each step. It highlights key programming concepts like function definition, loops, and conditionals, making it interactive with user input for endpoints and tolerance.